

Deep-Dive API Test Plan

The target endpoint selected for this section will be:

- PUT <https://reqres.in/api/users/{id}>

The objective will be to verify

- **Security:** validation of endpoint authentication
- **Data integrity:** validate that contract for this endpoint data is validated, and handles malformed requests properly.

1. Assumptions

As a previous step, the user created an account in reqres and obtained a x-api-key to use in the requests and be properly authenticated using its value in the header for valid authenticated scenarios.

Reqres has a mocked response, but I will assume it will behave as a correctly implemented endpoint following REST standards would behave. This includes PUT requests that should have the complete entity (not just a field like a PATCH), and immutable fields such as id or createdAt can't be modified by the user.

2. Technical risks

As Besides the negative scenarios that will be detailed below, we should consider other non functional requirements such as:

- Response time
- Proper status responses
- Data is properly stored (we could verify with a GET after a successful PUT)

3. Testing scope

Positive scenarios

- **Valid PUT - Non ASCII characters:** Full body with valid data. We should obtain a 200 OK response, and the updatedAt field should be correct.
- **Valid PUT - ASCII characters:** Same as above, but using special characters such as "ñ"

Negative scenarios - authentication

- **Unauthorized PUT - No token:** Full body with valid data, but no x-api-key header. Should return a 401 Unauthorized
- **Unauthorized PUT - Invalid token:** Full body with valid data, but with an invalid x-api-key header. Should return a 401 Unauthorized

Negative scenarios - Id parameter

We assume that all requests below have a valid x-api-key.

Id not found PUT: Full body with valid data, but with a non-existing id. Should return a 404 Not found

Invalid Id PUT: Full body with valid data, but with an invalid id (i.e.: not numeric id). Should return a 400 Bad Request

Negative scenarios - Wrong body

We assume that all requests below have a valid x-api-key.

- **Invalid body PUT - Empty body:** Empty body. Should return a 400 Bad Request.
- **Invalid body PUT - Immutable field:** Full body with valid data, including id or createdAt. Should return a 400 Bad Request.
- **Invalid body PUT - invalid type:** Full body with invalid data type (such as a number on first_name). Should return a 400 Bad Request
- **Invalid body PUT - invalid email:** Full body with valid data, but an invalid formed email. Should return a 400 Bad Request

We could perform more checks on each attribute of the body, as we did with email.

4. Automation considerations

The structure of this API test project could follow a service approach, in which the logic for each endpoint would be grouped by family (users in our case), to later be used in a fixture (by itself or along other endpoint families for more complex tests).

In this simple scenario we would have three layers:

- Service / Client layer, with the endpoint / test logic
- Fixture, where previous services would be injected
- Spec files, where fixture would be used to describe our scenarios

Additionally we would probably need data folders with json expected responses, and other helpers classes in a utils folder. I followed a similar approach (but oriented to pages) in Part 3, so more details can be found there.

To reduce the flakiness, we should always aim to revert any change we do during the testing, to leave the scenario as it was when we started. We could follow two approaches for this example:

- After a successful PUT, we should leave the user to its previous state, with a new PUT
- Or even better, if we can, create a user first with a POST, run our PUT tests with the recently created user, and then DELETE it

Besides this common practices, retries and waits (always within our non functional requirements) should be applied.

For traceability, debugging and observability we could use Playwright traces, and deploy in our pipeline all the artifacts needed to review the test execution. In this particular exercise, I also used GitHub Pages. More details in Part 3 exercise.